`requests`

Status: not covered by Software Lab-1 practicals **Where it is used:** `scripts/fetch_datasets.py` (`download`, `main`) — one `requests` call in the entire codebase, at line 37 **Closest lab concept:** Practical 4, functions and files — a download *is* a Lab 4 read-and-write loop; the only new thing is that the bytes arrive over a network instead of from disk

---

1. What it is

`requests` is a third-party HTTP client for Python. It turns "go and fetch this URL" into one function call, and it hands back an object holding the status code, the headers and the body.

```
pip install requests
```

It is **not** stdlib — unlike `os`, `csv`, `json` and `datetime`, which Practical 4 and its neighbours taught you with no install step. That matters for this project: the requirement files have to list it, and a grader running on a fresh machine needs `pip install -r` before the scripts work.

It appears in the course syllabus on the web-scraping line — "*application development to scrape the web with the help of standard libraries like Requests and bs4*". The project's own docstring points at that line:

```
"""Download candidate clinical datasets for Meridian.

Taught concepts used: os (P4), requests (syllabus - web scraping line).
Every file is public and requires no authentication.
"""
```

The last sentence is the design constraint that shapes the whole file: no API keys, no login, no tokens, no `.env`. That is why the script can be committed, re-run by anyone, and read aloud in a lab viva without exposing a credential.

Note what the syllabus says: "*standard libraries like Requests*" — the wording is loose, because `requests` is not in the standard library. The name is a library, the line is about libraries generally.

---

2. Why the project needed it

The brief asks for analytics over **real** clinical cohorts rather than toy lab data, because the interesting behaviour only shows up in real data: real missing-value markers, real class imbalance (500 non-diabetic against 268 diabetic), real columns whose units do not match. Practical 6's lab CSVs were numeric and clean, and this project explicitly notes that (`docs/concepts/pandas-missing-values.md`): without `na_values`, a dataset full of `?` markers reads every column as text and every numeric count comes back zero.

So the datasets had to be downloaded. And here is the honest gap: **nothing in Practicals 0-6 touches the internet.** The closest thing you have done is `open()` on a file that was already on your disk, read it, and write another one. There is no HTTP, no status code, no timeout, and no notion of a server that may simply not answer.

That gap is the entire reason this file exists, and the entire reason `requests` is in the syllabus. The datasets that make the whole project worth doing live on someone else's server:

```
SOURCES = {
    "pima_diabetes.csv":
        "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv",
    "heart_disease_uci.csv":
        "https://raw.githubusercontent.com/jbrownlee/Datasets/master/heart-statlog.csv",
    "breast_cancer_wisconsin.csv":
        "https://raw.githubusercontent.com/jbrownlee/Datasets/master/breast-cancer-wisconsin.csv",
    "ionosphere.csv":
        "https://raw.githubusercontent.com/jbrownlee/Datasets/master/ionosphere.csv",
    "sonar.csv":
```

```
        "https://raw.githubusercontent.com/jbrownlee/Datasets/master/sonar.csv",
    "pima_diabetes_local.csv":
        "https://raw.githubusercontent.com/plotly/datasets/master/diabetes.csv",
}
```

A dict of `local_filename -> URL`, six entries. The filenames on the left are what the rest of the project expects; the URLs on the right are somebody else's server. Five point at one repository (`jbrownlee/Datasets`) and one at another (`plotly/datasets`) — see section 5 for why that inconsistency matters.

Everything downstream is local. `backend/analytics.py` reads from `data/datasets/` and never opens a URL, which is the right separation: the network is touched once, by one script, and the rest of the project is deterministic and testable offline.

---

3. The four pieces actually used

The whole of `scripts/fetch_datasets.py`, `download` verbatim (`scripts/fetch_datasets.py:28-47`):

```
def download(filename, url):
    """Fetch one CSV and save it into the raw folder."""
    path = os.path.join(RAW_DIR, filename)

    if os.path.exists(path):
        size = os.path.getsize(path)
        print(f"  skip  {filename}  (already here, {size} bytes)")
        return True

    response = requests.get(url, timeout=30)

    if response.status_code != 200:
        print(f"  FAIL  {filename}  HTTP {response.status_code}")
        return False

    with open(path, "w", newline="") as f:
        f.write(response.text)

    print(f"  ok    {filename}  {os.path.getsize(path)} bytes")
    return True
```

Twenty lines. Four of them are `requests`; the rest is Practical 4.

3.1 `requests.get(url, timeout=30)`

```
    response = requests.get(url, timeout=30)
```

One function, one URL, one keyword argument. It performs the HTTP request and returns a **response object** — not a string. The body, the status code and the headers all hang off that object, which is why the next two sections are properties of a return value rather than more function calls.

The `timeout=30` is the subject of section 4 and is the single most important line in the file.

3.2 The status code check, before anything else

```
    if response.status_code != 200:
        print(f"  FAIL  {filename}  HTTP {response.status_code}")
        return False
```

The most important lesson about `requests` is that a `requests.get()` call does not raise when the request fails. This is the difference between `requests` and `urllib`, which raises `HTTPError` on a 404. With `requests`, a missing file, a renamed repository, a typo in a URL and a typo in a filename are all *completely normal returns*. You get a response object with `status_code` of 404 and a body containing `"404: Not Found"`, and your program carries on as if it had succeeded unless you check.

`status_code` is the three-digit HTTP result code. The numbers you will meet downloading files:

Code	Meaning	What it means here
200	OK	got it — the only code this script accepts
301 / 302	Moved / Found	the URL moved; `requests` follows these by default
403	Forbidden	the host is refusing you, often a bot filter
404	Not Found	the file is not there. Most common failure.
500	Server Error	their problem, not yours
timeout	*(exception, not a code)*	see section 4

The script prints the actual code rather than a generic message, and returns `False` so `main()` can count the failures. Being specific in the error message is what made section 5 diagnosable.

3.3 Why the check comes *before* `response.text`

This ordering is the whole design of the function, so it is worth being precise about. The naive version checks later, or not at all. **The next block is the counter-example, written out for comparison — it is not code that exists in this repository**; the real `download()` is quoted in full in section 3:

```
    response = requests.get(url, timeout=30)
    with open(path, "w", newline="") as f:
        f.write(response.text)
```

— check later, or not at all. It looks fine. It is wrong in four ways at once.

1. **A 404 body is a valid string.** `"404: Not Found"` is text. It will write. It will write successfully. The file on disk will contain the words "404: Not Found" and nothing else, and `open()` will not complain, because from the filesystem's point of view the write succeeded perfectly.
2. **The damage is permanent and silent.** Once that file exists, the skip-if-exists guard in 3.5 refuses to re-download it. One bad run poisons the script until a human goes and deletes the file by hand.
3. **`f.write()` failing is not a status check.** You would be relying on an `IOError` to detect a *server-side* problem. Wrong layer: the disk was fine. The problem was upstream, and only `status_code` knows that.
4. **The next stage cannot tell.** `backend/analytics.py` opens these files with `pd.read_csv`. A file containing `404: Not Found` produces a one-column dataframe with a nonsense header, or raises deep inside pandas — an error message pointing at the wrong project entirely.

The rule this encodes: **check that the server said yes before you trust anything it sent.** The status code is a statement about the *request*; the body is a statement about the *content*. Only the first one tells you whether the second one is worth reading. Verified behaviour for a genuine 404 today:

```
404  True  heart_disease_uci.csv
```

The second field is "did the body contain any bytes" — and it is `True`. A 404 hands you a body. It just is not the file.

3.4 `f.write(response.text)` — and why `.text`, not `.content`

```
    with open(path, "w", newline="") as f:
        f.write(response.text)
```

`response.text` is the body **already decoded into a Python string**; `response.content` is the same body as raw `bytes`. The file is written in text mode `"w"`, so `response.text` is the correct pairing. Using `.content` here would require `"wb"` and would skip the decoding step.

`newline=""` is a Practical 4 / `csv` habit — see `docs/concepts/csv-module.md` — and it is worth keeping: it

stops Windows from translating every `\n` into `\r\n` as it writes. The files land byte-identical to what the server sent.

There is a real limitation hiding here, and it explains a design decision elsewhere in the project: **`.text` is for text.** Decoding a PNG or a ZIP into a string and writing it back out corrupts it. Any dataset served as a compressed archive cannot be saved by this function as written — which is exactly what happened with the UCI downloads (section 6).

3.5 The skip-if-exists guard

```
    if os.path.exists(path):
        size = os.path.getsize(path)
        print(f"  skip  {filename}  (already here, {size} bytes)")
        return True
```

Practical 4 again: `os.path.exists`, `os.path.getsize`, `os.path.join`.

Three reasons this guard earns its place, in increasing order of importance:

- **Speed.** 6 files, ~600 KB total. Irrelevant. Be honest about this one.
- **Politeness.** The upstream is a public repository. Re-downloading files you already have is rude, and hammering somebody's GitHub raw endpoint in a loop is how you get rate-limited or blocked.
- **Correctness — this is the real reason.** Section 3.3 explained that a failed download can leave a file behind. This guard means the script is **not idempotent by accident**: once a good copy is on disk, re-running cannot clobber it with a fresh failure. It returns `True`, because "already here" *is* success from the caller's point of view.

That third point is the general lesson, and it is easy to get backwards: a "skip if exists" check is not primarily a performance optimisation, it is a **claim about what is already on disk**. It is only safe if everything that wrote there was validated first. Here, section 3.2 is what makes the claim true.

3.6 `main()` — the loop that counts

```
def main():
    if not os.path.exists(RAW_DIR):
        os.makedirs(RAW_DIR)

    print(f"Downloading into {RAW_DIR}")
    results = []

    for filename in SOURCES:
        results.append(download(filename, SOURCES[filename]))

    ok = results.count(True)
    print(f"\n{ok} of {len(results)} datasets ready")
```

Practical 1 loops and lists. `results` collects one `True`/`False` per file, and `results.count(True)` — the same `.count()` you would use on a list of anything — reports how many succeeded. One failure does not stop the other five, which is the correct behaviour for a batch fetch: you want the four working files *and* a clear report that the fifth failed, not an exception that abandons the run.

`os.makedirs` is guarded by `os.path.exists` because `os.makedirs` without `exist_ok=True` raises `FileExistsError` on a second run, and this script is designed to be run repeatedly.

---

4. Gotcha: `timeout` is mandatory, not optional

```
    response = requests.get(url, timeout=30)
```

Write this argument on every single `requests` call, forever. It is not a tuning knob.

`requests` has **no default timeout**. That is deliberate on the library's part — a library that guesses how long a legitimate slow download should be allowed to take would break real users. The consequence is that `requests.get(url)` with no timeout waits **indefinitely**, and the socket's own OS-level timeout is measured in *minutes*.

Here is the failure this project actually hit during development, and the shape it takes:

1. A URL in `SOURCES` is wrong. Not misspelled-wrong — *stale* wrong. The repository was reorganised and the file is no longer at that path.
2. `requests.get(url)` is called with no timeout.
3. Depending on how the host reacts, you get one of two outcomes, and **both are bad**: - the host answers `404` quickly and everything is fine (lucky), or - the connection is routed somewhere that accepts the TCP connection and then says nothing at all. No headers, no body, no error. Ever.
4. The script hangs. Not "slowly" — *hung*. The prompt never returns. The files after the bad one in the `SOURCES` dict are never attempted.

The cruel part is that outcome (b) looks like a Python problem. You start debugging your `for` loop, your dict, your string concatenation. You add `print()` statements. They all execute fine. Nothing is wrong with any of it. The process is blocked on a `recv()` inside the HTTP library, waiting for a server that has nothing to say.

This is a **hang, not a crash**, and that is what makes it expensive. A crash gives you a traceback and a place to start. A hang gives you nothing — no message, no exit code, no log line, and if it is the Flask server or a test run rather than a script, it takes the whole process down rather than one function.

With `timeout=30`, the same situation raises `requests.exceptions.Timeout` after 30 seconds and names itself. You still have a bug — a wrong URL — but you have a *diagnosable* one.

Two habits worth forming from this:

- **Always pass `timeout`.** No exceptions, no "I'll add it if it feels slow". Thirty seconds is generous for a small CSV and costs nothing when the host is healthy.
- **`timeout` is not `requests.exceptions.RequestException`.** Only `status_code` catches the HTTP-level failures. A timeout never produces a status code at all, because no response arrived. A production-grade fetcher wraps the call in `try/except requests.exceptions.RequestException` so both failure families are handled. **This project does not**, and that is a genuine gap: `download()` returns `False` for a 404 but lets a `Timeout` propagate out of `main()` and kill the run. It is left unhandled deliberately to keep the lab code short and readable, and naming it here is more useful than quietly pretending it is robust.

The related trap: `timeout` is not a total-deadline. It applies separately to connecting and to reading between bytes. A server that dribbles one byte every 29 seconds keeps a 30-second timeout happy forever. For a small CSV that is a theoretical worry; for a large download it is a real one, and the answer is a loop with a wall-clock budget, which this script does not need.

---

5. Real limitation: hardcoded URLs rot

This project got this wrong, repeatedly, and it is not fixable by writing better code. It is a property of the internet.

The honest history, as far as the repository records it: the working approach was to treat the URL list as a *candidate* list, test each entry with a quick HTTP request, and keep only the ones that responded. The function is even named after that framing — it fetches **candidate** datasets, and `main()` prints a scoreboard rather than an assertion:

```
Downloading into data/datasets/raw
  skip  pima_diabetes.csv  (already here, 528 bytes)
  FAIL  heart_disease_uci.csv  HTTP 404
  ok    breast_cancer_wisconsin.csv  111 KB
  ...
5 of 6 datasets ready
```

What I measured. Running the six URLs through `requests.get` with a 30-second timeout just now:

```
200  True  pima_diabetes.csv
404  True  heart_disease_uci.csv
200  True  breast_cancer_wisconsin.csv
200  True  ionosphere.csv
200  True  sonar.csv
200  True  pima_diabetes_local.csv
```

One of six is a 404 today — `heart-statlog.csv` no longer exists at that path in `jbrownlee/Datasets`. During development the figure reported was 5 of 6. I could not find that figure recorded anywhere in the repository, so treat it as reported rather than verified — but the point survives either way, and is arguably made *better* by the discrepancy: **the failure count is not a stable property of anything. It changes as repositories reorganise.** A script that was 5-of-6 broken in one month can be 1-of-6 broken in the next, with no edit to the code at all.

This is normal and it is not negligence on anyone's part. A GitHub repository is a living thing: branches get renamed, files get moved into subdirectories, datasets get replaced with newer versions, whole collections get archived. A URL is a promise about a location on someone else's disk, made in advance, with no version and no guarantee.

What the repository shows. `data/datasets/raw/` contains four files:

```
heart_disease_processed.csv
mammographic_masses.csv
parkinsons.csv
pima_diabetes_local.csv
```

Three of those four filenames **do not appear anywhere in the current `SOURCES` dict**. Only `pima_diabetes_local.csv` matches. Those three are exactly the files `scripts/stage_datasets.py` still looks for:

```
def stage_heart():
    """Cleveland heart data: no header, 14 columns, '?' means missing."""
    src = os.path.join(RAW, "heart_disease_processed.csv")
```

So the evidence in the tree is unambiguous: `SOURCES` was written at least twice. An earlier version listed the UCI-derived URLs for heart disease, mammographic masses and parkinsons; those files were really downloaded (they are on disk, and they are what `data/datasets/pima_diabetes.csv` and its siblings are built from); then the dict was rewritten around GitHub raw copies, and the old URLs were deleted rather than left to rot visibly. The current script is a **sweep for additional candidate datasets** — the five `jbrownlee/Datasets` entries plus a Plotly copy of Pima — not the provenance of the four shipped datasets.

Why the `status_code` check is therefore not optional politeness. It is the *only* thing standing between this file and a directory of files named confidently after datasets that do not exist. And section 3.5 is what stops a bad answer becoming permanent: because a 404 never reaches `f.write`, the skip-if-exists guard never sees a poisoned file, so the next run simply tries again.

The habits worth taking from this:

- **Verify a URL before you depend on it.** One `requests.get` with a short timeout, check the code, and only then write it into a source file. Cheap, and it converts an hour of debugging into a one-

line test.

- **Prefer a stable host over a convenient one.** These are mirrors of datasets, not the publishers. The UCI Machine Learning Repository is the canonical source and numbers its datasets with permanent ids; a personal GitHub collection is one person's folder.
- **Expect to fix the list, and keep the failures visible.** A 404 logged as `FAIL heart_disease_uci.csv HTTP 404` is a five-second fix. A silently empty dataset is a mystery.

---

6. Correction: the UCI `.zip` route is *not* in this file

This section exists because the natural guess about this script is wrong, and the wrong guess is well-supported by the rest of the repository.

The obvious story is that `parkinsons.csv` and `mammographic_masses.csv` came from `https://archive.ics.uci.edu/static/public/<id>/<name>.zip` and needed `zipfile` to extract. That story is **half true**, and the half that is false matters.

What is true. `scripts/stage_datasets.py:49` defines a real UCI archive endpoint:

```
PARKINSONS_URL = "https://archive.ics.uci.edu/static/public/174/parkinsons.zip"
```

and `scripts/stage_datasets.py:10` imports the module that could open it:

```
import zipfile
```

What is false. `PARKINSONS_URL` is **never used by any function**, and `zipfile` is **never called** — it is the only occurrence of the word in the entire codebase. `stage_parkinsons()` expects a plain `raw/parkinsons.csv` to already be on disk:

```
def stage_parkinsons():
    """Parkinsons already ships a header row; just tidy the column names."""
    src = os.path.join(RAW, "parkinsons.csv")
    if not os.path.exists(src):
        print("  skip  parkinsons (run fetch first)")
        return
```

The archive-handling design was planned and then **dropped in favour of flat CSV downloads**. `docs/concepts/shutil-and-zipfile.md` §6.2 reaches the same conclusion and says it bluntly: "`zipfile` *is imported as a record of an intended design, not as working code. An unused import is a plan, not a feature.*"

So the honest version of the story, in the order it actually happened:

1. The plan was to pull datasets from UCI as `.zip` archives. Those archives contain `.data` files — whitespace-separated, **no header row**, `?` for missing — which is not a CSV by any reasonable definition, and Practical 6's `pd.read_csv` would read the first data row as the column names.
2. That is why `stage_datasets.py` exists at all, and why it writes its own header from the official column lists before handing anything to pandas. `zipfile` was the missing piece of that plan.
3. The archive step was dropped. `fetch_datasets.py` fetched flat `.csv` files from GitHub raw URLs instead, which needed no unzipping.
4. The `zipfile` import and `PARKINSONS_URL` constant were left behind. The staged CSVs in `data/datasets/` were produced during step 3, and their *contents* came from the UCI collection — but not through a `.zip` that this codebase ever opened.

Two lessons, both about the archaeology of your own code:

- **A URL constant is not a download.** Reading `PARKINSONS_URL` and concluding "this project downloads from UCI" is exactly the error an unused import invites.

- **The leftover import is the fossil.** If you want to know what a project *intended*, imports and unused constants are the record. If you want to know what it *does*, follow the call graph — in this case from `stage_parkinsons` back to a file on disk.

---

7. Why `bs4` is not here, and would be the wrong tool

The syllabus sentence names two libraries: "*standard libraries like Requests and bs4*". This project uses the first and not the second, and the reason is worth more than the library.

`bs4` (Beautiful Soup) parses HTML. It turns a document — an HTML page — into a tree you can walk and search, so you can pull values out of `<table>` markup, `<div>`s and link `href`s. It exists because HTML is structured for *humans reading it in a browser*, not for programs.

None of the six targets is HTML. They are `.csv` files, served as `text/csv` or `text/plain`. The structure we want is already there — rows and columns — and the correct parser for that is Practical 6's `pd.read_csv`, which the project uses immediately afterwards in `analytics.read_dataset`:

```
    return pd.read_csv(path, na_values=MISSING_MARKERS, keep_default_na=True)
```

Running `bs4` over a CSV would be a category error. It would hand back one giant text node containing the whole file, and we would then have to re-implement `split(",")` — which is *less* correct than `csv.reader`, because it breaks on quoted fields containing commas. So: parse CSV with `csv` or pandas, never with an HTML parser.

Where `bs4` would genuinely be right, and where this project deliberately did not go:

- A dataset published only as an HTML table on a university webpage.
- A site with no CSV or JSON export, requiring you to walk `<tr>`/`<td>`.
- Following pagination links to collect many pages.

None of those applied. Every dataset here has a machine-readable file behind a stable URL, and the brief is analytics, not crawling. Adding `bs4` would have added an install and a parsing failure mode for zero gain.

The related judgement, since it is the one beginners get wrong: **HTML scraping is also the ethically and legally loaded option.** Many sites have terms that forbid it, and automated fetching can be a denial of service on a small host. Downloading a published CSV is not scraping — it is fetching a file the publisher put there to be fetched. That distinction is why the file's docstring can say "*Every file is public and requires no authentication*" and mean it as a design guarantee.

---

8. Questions this should answer

- `requests.get()` did not raise, yet the download failed. What did it return instead, and which attribute exposes the problem?
- Why must `status_code` be checked *before* `f.write(response.text)`? What specifically goes wrong if a 404 body is written to disk, and which later line makes that mistake permanent?
- Why is `timeout=30` mandatory on every call rather than a performance tweak? Describe the failure mode it prevents and why a hang is more expensive to debug than a crash.
- Four files sit in `data/datasets/raw/` whose names are absent from the current `SOURCES` dict. What does that tell you about the script's history, and what does it say about hardcoded URLs?
- `PARKINSONS_URL` is a real UCI archive endpoint and `zipfile` is imported in `stage_datasets.py`, yet neither is ever used. What does an unused import tell you about a codebase?
- The syllabus pairs `requests` with `bs4`. Why is `bs4` the wrong tool for downloading a CSV, and what should parse CSV instead?